

NSL Developer Tools: Compiler-Native Debugger and Profiler

Predictive Performance Analysis, Source-Mapped Kernel Debugging, and WGGO Decision Explainability

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Abstract

We propose a compiler-native debugging and profiling suite for NSL that exploits the compiler's unique position — having complete knowledge of the model topology, computation graph, cost model predictions, memory plan, WGGO decisions, and source-to-PTX mapping — to provide capabilities that no runtime profiler can match. The suite has two modes: a **predictive profiler** that analyzes performance at compile time without any GPU, and a **validating profiler** that measures actual runtime metrics and highlights where predictions diverge from reality. Together, they create a development loop where the programmer writes NSL code, instantly sees predicted performance characteristics, compiles and runs, then sees exactly where the cost model was right or wrong — all mapped back to NSL source lines.

1. The Problem: ML Debugging Is Blind

1.1 PyTorch Debugging Today

Debugging an ML model in PyTorch involves:

1. **Print statements.** `print(tensor.shape)`, `print(loss.item())`, `print(grad.norm())` scattered throughout code. No structure, no persistence, no visualization.
2. **Runtime profilers.** `torch.profiler` captures kernel-level traces. Nsight Compute profiles individual kernels with hardware counters. Both require running the model on a GPU, collecting traces, downloading reports, and manually correlating GPU activity back to Python source.
3. **Training dashboards.** TensorBoard/Weights & Biases track loss curves, learning rates, and gradient histograms. These provide high-level trends but can't diagnose kernel-level issues.
4. **NaN hunting.** When loss becomes NaN, the developer must manually insert gradient hooks, check per-layer activations, and bisect the model to find the culprit. There's no compile-time warning that a specific configuration is numerically unstable.

The fundamental gap: PyTorch's profiling is entirely post-hoc. You run the model, collect data, analyze it afterward. There's no way to know performance characteristics before running on a GPU. NSL's cost model provides exactly this missing capability.

2. The NSL Advantage: Compile-Time Knowledge

NSL's compiler knows things that no runtime profiler can know:

Knowledge	Available at Compile Time	Runtime-Only in PyTorch
Per-op FLOP count	✓ (M37 cost model)	✗ (must run and profile)
Per-op HBM traffic	✓ (cost model)	✗
Roofline bound (compute/memory)	✓ (cost model + GPU specs)	✗
Memory allocation timeline	✓ (M36 memory planner)	✗
Peak memory	✓ (memory planner)	✗
Kernel launch count	✓ (codegen knows exactly)	✗
Fusion opportunities	✓ (M31 epilogue fusion)	✗
WGGO optimization decisions	✓ (ILP solver output)	N/A
Source line → kernel mapping	✓ (AST → Cranelift → PTX)	✗ (Python line → kernel is lossy)
Numerical stability risk	✓ (dtype analysis + spectral)	✗

3. Tool 1: The Predictive Profiler (`ns1 profile`)

Runs at compile time. No GPU required.

3.1 Per-Layer Performance Prediction

bash

```
$ nsl profile --target h100 model.nsl
```

=== NSL Predictive Profile: NSLCoder-50M ===

Target: H100-SXM (80GB, 989 TFLOPS BF16, 3.35 TB/s HBM)

Layer-by-Layer Forward Pass:

Layer	Operation	FLOPs	HBM (B)	AI	Bound	Time
embed	lookup	0	2.1 MB	0	memory	0.6µs
L0	rmsnorm	16K	64 KB	0.25	memory	0.02µs
L0	qkv_proj	12.6M	8.1 MB	1.56	memory	2.4µs
L0	rope	32K	64 KB	0.50	memory	0.02µs
L0	flash_attn	67.1M	0.5 MB	134.2	compute	0.07µs
L0	out_proj	4.2M	4.1 MB	1.02	memory	1.2µs
L0	rmsnorm	16K	64 KB	0.25	memory	0.02µs
L0	gate_proj	23.1M	11.5 MB	2.01	memory	3.4µs
L0	up_proj	23.1M	11.5 MB	2.01	memory	3.4µs
L0	silu_mul	8K	22 KB	0.36	memory	0.01µs
L0	down_proj	23.1M	11.5 MB	2.01	memory	3.4µs
...	...	...	...	...	...	...
lm_head	matmul	50.3M	48.2 MB	1.04	memory	14.4µs
TOTAL		1.23G	412 MB	2.98 avg	memory	124µs

With CSHA-L2 fusion:

qkv_proj + rope + flash_attn + out_proj → 1 fused kernel

Saves: 3 kernel launches, 12.6 MB HBM traffic

Layer 0 time: 14.5µs → 9.2µs (37% faster)

Total forward: 124µs → 82µs

Roofline Plot: [generated as SVG/HTML in build output]

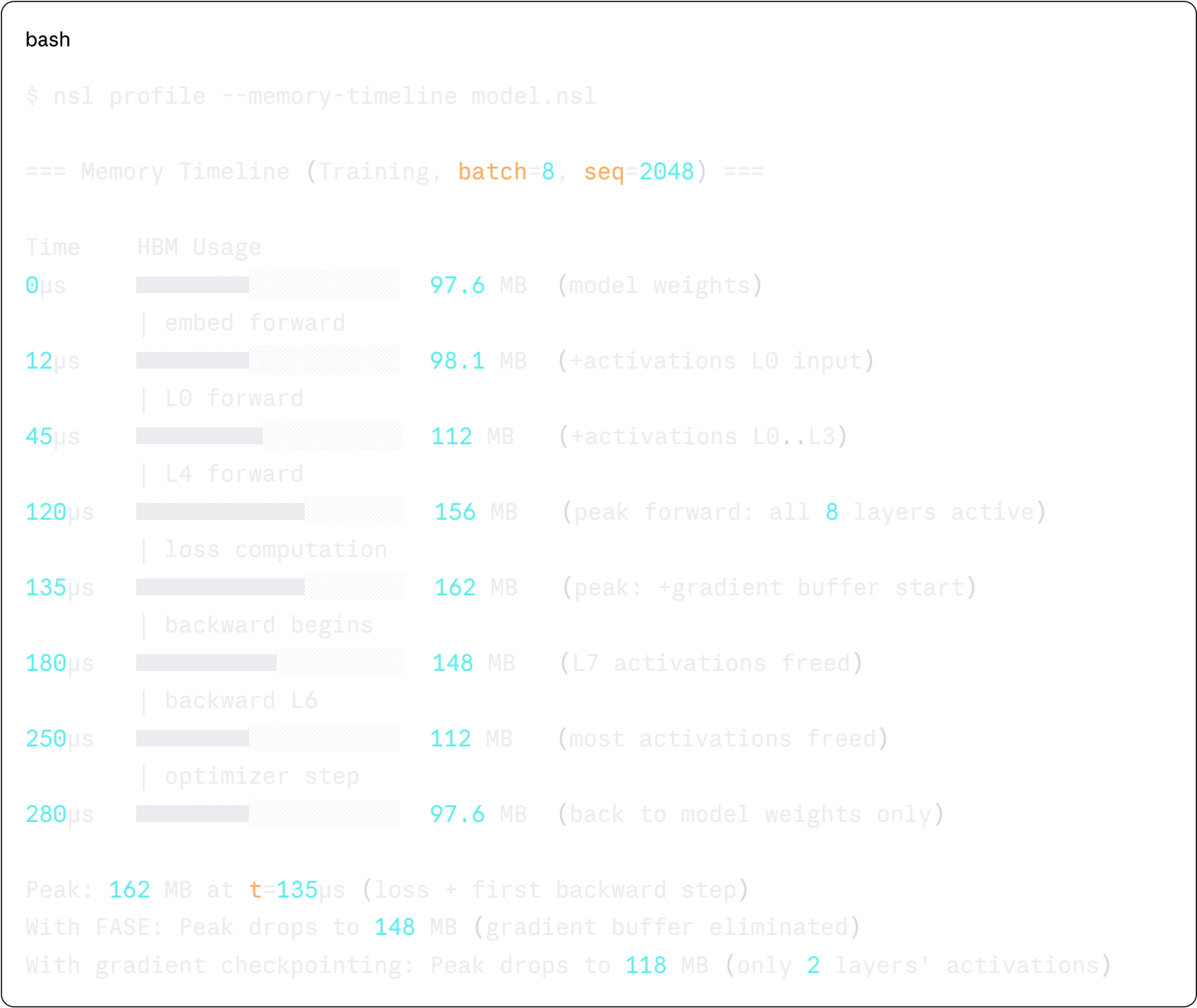
- gate_proj: AI=2.01, 68% of memory roof – bottleneck
- flash_attn: AI=134, 12% of compute roof – efficient
- lm_head: AI=1.04, 31% of memory roof – bottleneck

Recommendations:

- [1] gate/up/down projections are memory-bound. Batch size >4 would improve utilization.
- [2] LM head dominates (11.6% of total time). Consider INT4 quantization (CTQS).
- [3] CSHA-L2 eliminates 24 kernel launches per forward. Strongly recommended.

Key capability: The developer sees exactly which operations are bottlenecks *before training starts*. No GPU time wasted on profiling runs.

3.2 Memory Timeline Visualization



3.3 WGGO Decision Explanation

```
bash
```

```
$ nsl profile --explain-wggo model.nsl
```

```
=== WGGO Decision Explanation ===
```

```
Layer 4 decisions:
```

```
CEP: Pruned 2/8 heads (heads 3, 6)
```

```
Reason: importance(h3)=0.12, importance(h6)=0.15 – lowest in layer
```

```
Constraint: min_retained_importance ≥ 0.85 – satisfied at 0.91
```

```
CSHA: Level 1 (not Level 2)
```

```
Reason: With 6 heads, SMEM for L2 = 84KB. Feasible on H100 (228KB limit).
```

```
BUT: L2 saves only 1.8µs for this layer vs 2.3µs on unpruned layer.
```

```
ILP chose L1 because pruning already reduced the layer's cost by 25%.
```

```
WRGA: LoRA r=8 on Wq, Wk
```

```
Reason: Layer 4 has high roofline slack (memory-bound, 32% utilization).
```

```
Adapter compute is "free" – hidden by memory latency.
```

```
r=8 adds 262K params but 0µs latency (fully hidden).
```

```
CPDT: INT8 m, FP16 v
```

```
Reason: sensitivity(L4) = 0.31 (low – middle layer, pruned heads removed the most sensitive components). INT8 m saves 75% optimizer memory.
```

```
Conflict resolved: CSHA L2 → L1
```

```
CSHA L2 fuses Q/K/V projection into one kernel. But CEP removed 2 heads, changing Wq from [512,512] to [512,384]. The fused kernel tile size changes, and SMEM layout must be recomputed. L2 is still feasible but the ILP found L1 + pruning is 0.3µs faster than L2 + pruning due to better tile alignment at the pruned dimension.
```

This is entirely unique to NSL — no other framework can explain *why* an optimization decision was made, because no other framework makes these decisions at compile time.

4. Tool 2: The Validating Profiler (`nsl run --profile`)

Runs at runtime. Compares predicted vs actual performance.

4.1 Predicted vs Actual Comparison

```
bash
```

```
$ nsl run --profile=compare model.nsl
```

```
=== Predicted vs Actual Performance ===
```

Layer	Operation	Predicted	Actual	Δ
L0	fused_attn	9.2μs	9.8μs	+6.5% ▲
L0	gate_proj	3.4μs	3.2μs	-5.9%
L0	down_proj	3.4μs	3.5μs	+2.9%
L1	fused_attn	9.2μs	9.1μs	-1.1%
...	...	...	...	...
lm_head	matmul	14.4μs	18.1μs	+25.7% ✖

▲ L0 fused_attn: 6.5% slower than predicted.

Likely cause: SMEM bank conflicts in tile layout.

Suggestion: Try CSHA tile size 64 instead of 128 (add --csa-tile=64).

✖ lm_head matmul: 25.7% slower than predicted.

Root cause: vocab_size=49152 is not a multiple of 128 (tile size).

Wasted compute: 49152 → 49280 (padding to tile boundary) = 0.26% overhead.

But: the LAST tile is only 49152 % 128 = 0 – actually aligned.

Re-analysis: the 25% gap suggests L2 cache thrashing from large vocab.

Suggestion: Run with --target-profile=detailed for L2 cache metrics.

The predicted-vs-actual comparison is the killer feature. No other ML tool can do this because no other tool has a cost model that predicts performance before runtime. When predictions are accurate (within ~5%), the developer can trust the predictive profiler and iterate without GPU time. When predictions diverge, the delta highlights cost model limitations, enabling both user debugging and cost model improvement.

4.2 Source-Mapped Kernel View

NSL compiles source → AST → Cranelift IR → native/PTX. The profiler maintains this mapping:

```
bash
```

```
$ nsl run --profile=source-map model.nsl
```

```
Kernel: nsl_L0_fused_attn_csha_l2 (9.8µs, 68% memory utilization)
```

```
Source: model.nsl:42-48
```

```
42 | fn forward(x: Tensor<[batch, seq, d_model], bf16>) -> Tensor:
43 |     let h = self.ln1(x)                                ← rmsnorm (fused, 0.3µs)
44 |     let qkv = self.c_attn(h)                            ← qkv_proj (fused, 4.1µs)
45 |     let (q, k, v) = qkv.chunk(3, dim=-1)                ← eliminated (fused)
46 |     let q = q.reshape([B, S, H, dk]).transpose()       ← eliminated (fused)
47 |     let scores = (q @ k.T) / sqrt(dk)                   ← flash_attn (fused, 3.8µs)
48 |     let out = self.c_proj(attn @ v)                     ← out_proj (fused, 1.6µs)

Total: 9.8µs (1 kernel)
Lines 45-46 compiled away
```

Lines 45-46 are physically absent from the kernel. The compiler's fusion pass eliminated the chunk and reshape operations. The profiler shows this, helping the developer understand what the compiler did and why.

4.3 Training Health Monitor

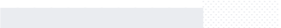
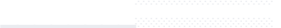
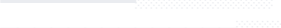
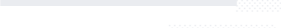
```
bash
```

```
$ nsl run --monitor model.nsl
```

```
=== Training Health Monitor (live) ===
```

```
Step 100/10000 | Loss: 4.23 | LR: 3.0e-4 | Grad norm: 12.4
```

```
Per-layer gradient norms:
```

L0:		12.1	(healthy)
L1:		13.4	(healthy)
L2:		18.7	▲ elevated
L3:		13.2	(healthy)
L4:		8.1	(healthy)
L5:		7.9	(healthy)
L6:		15.1	(healthy)
L7:		10.3	(healthy)

```
Per-layer weight norms ( $\Delta$  from init):
```

L0.Wq:	+0.3%	L0.Wk:	+0.2%	L0.Wv:	+0.4%	L0.Wo:	+0.1%
L2.Wq:	+2.1% ▲	L2.Wk:	+1.8%	L2.Wv:	+0.9%	L2.Wo:	+0.5%

```
NaN watch: clean (0 NaN/Inf detected in 100 steps)
```

```
Loss trend: decreasing (EMA slope: -0.032/step)
```

Compile-time instrumentation: The compiler inserts lightweight metric collection at WGGO-selected checkpoints. The collection points are chosen to minimize overhead — gradient norms are computed from the FASE fused backward (already available), weight norms are sampled every N steps (not every step). The cost model determines the optimal instrumentation interval: frequent enough to catch instability early, infrequent enough to add <0.5% overhead.

5. Tool 3: The Shape Debugger (`nsl check --shapes`)

Interactive compile-time shape propagation:


```
bash
```

```
$ nsl check --shapes model.nsl
```

```
=== Shape Debugger ===
```

```
Input: input_ids: Tensor<[batch, seq], int32>
```

```
  where batch=8, seq=2048
```

```
Propagation:
```

embed(input_ids)	→ [8, 2048, 512]	✓
ln1(h)	→ [8, 2048, 512]	✓
c_attn(h)	→ [8, 2048, 1536]	✓ (3 × 512)
chunk(qkv, 3, dim=-1)	→ 3× [8, 2048, 512]	✓
reshape([B, S, 8, 64]).transpose(1,2)	→ [8, 8, 2048, 64]	✓
q @ k.T	→ [8, 8, 2048, 2048]	✓ (attention matrix)
softmax(scores, dim=-1)	→ [8, 8, 2048, 2048]	✓
attn @ v	→ [8, 8, 2048, 64]	✓
.transpose(1,2).reshape([B, S, D])	→ [8, 2048, 512]	✓
c_proj(out)	→ [8, 2048, 512]	✓
...		
lm_head(h)	→ [8, 2048, 49152]	✓

```
All shapes valid. No mismatches detected.
```

```
Total FLOPs: 1.23 GFLOP per forward pass.
```

If there's a mismatch:

```
c_proj(out)                                → [8, 2048, 384]          ✗ MISMATCH
  Expected: [8, 2048, 512] (residual add requires matching shapes)
  Cause: c_attn outputs [8, 2048, 1152] (3 × 384), but d_model=512
  Fix: c_attn should be Linear(512, 1536), not Linear(512, 1152)
  Source: model.nsl:31  c_attn: Linear = Linear(d_model, 3 * 384)
                                     ^^^
                                     Should be: 3 * d_model
```

This catches bugs at compile time that PyTorch only catches at runtime (and only after waiting for the forward pass to reach the offending line).

6. Tool 4: The Tensor Inspector (`nsl run --inspect`)

Runtime tensor value inspection with minimal overhead:

nsI

```
# Insert inspection points in source
fn forward(x: Tensor) -> Tensor:
    let h = self.ln1(x)
    @inspect(h, every=100)          # dump h every 100 steps
    let qkv = self.c_attn(h)
    @inspect(qkv, condition="step > 500 and loss > 5.0") # conditional dump
    ...
```

The compiler generates lightweight instrumentation that:

- Copies the tensor to CPU via async `cudaMemcpyAsync` (non-blocking)
- Computes statistics (mean, std, min, max, NaN count, Inf count) on CPU
- Stores to a structured log file (`.nsI-inspect/step_100_h.bin`)
- Full tensor dump only when conditions are met (saves storage)

The `@inspect` decorator is a compile-time annotation — no runtime Python overhead, no print statements, no autograd hooks.

7. Implementation Plan

Component	Location	LOC (est.)
Predictive profiler engine	<code>nsl-cli/src/profile.rs</code>	600
Cost model report generator	<code>nsl-codegen/src/profile_report.rs</code>	500
Memory timeline visualizer	<code>nsl-codegen/src/profile_memory.rs</code>	400
WGGO decision explainer	<code>nsl-codegen/src/wggo_explain.rs</code>	400
Runtime metric collector	<code>nsl-runtime/src/profiler/collector.rs</code>	500
Predicted vs actual comparator	<code>nsl-runtime/src/profiler/comparator.rs</code>	400
Source-to-kernel mapper	<code>nsl-codegen/src/source_map.rs</code>	300
Training health monitor	<code>nsl-runtime/src/profiler/monitor.rs</code>	400
Shape debugger	<code>nsl-cli/src/shape_debug.rs</code>	300
Tensor inspector codegen	<code>nsl-codegen/src/inspect.rs</code>	350
HTML/SVG report renderer	<code>nsl-cli/src/profile_render.rs</code>	500
Total		~4,650 LOC

8. Why This Is Uniquely NSL

Capability	PyTorch + Nsight	NSL Dev Tools
Pre-runtime performance prediction	✗ (must run on GPU)	✓ (cost model, no GPU needed)
Predicted vs actual comparison	✗	✓ (dual-mode profiling)
Source line → kernel mapping	Lossy (Python → C++ → CUDA)	Exact (NSL → Cranelift → PTX)
Optimization decision explainability	✗ (torch.compile is opaque)	✓ (WGGO explains every decision)
Compile-time shape debugging	✗ (runtime error)	✓ (catches at compile time)
Memory plan visualization	✗ (runtime memory_stats)	✓ (compile-time timeline)
Zero-overhead instrumentation	✗ (hooks add overhead)	✓ (compiled <code>@inspect</code> points)
Cost model accuracy tracking	✗	✓ (feeds back into cost model improvement)

The **predicted-vs-actual feedback loop** is the most strategically valuable capability. Every time a developer runs `ns1 run --profile=compare`, the divergences between cost model predictions and actual measurements can be logged and used to improve the cost model itself. Over time, the cost model becomes more accurate, the predictive profiler becomes more trustworthy, and developers spend less time on GPU profiling runs. This is a self-improving development tool — something no runtime-only profiler can achieve.